

Quantum Resource Management Interface (QRMI) and WLM Integration

Munetaka Ohtani
QRMI Maintainer, Senior Software Engineer
IBM Quantum



Highlights

We introduce a unified resource management solution that delivers the **Quantum Resource Management Interface (QRMI)** to abstract vendor-specific quantum hardware and uses the **SPANK plugins** framework to control hybrid classical–quantum workflows in Slurm.

- Native QPU integration with Workflow Manager - **Slurm**
- Unified job submission for quantum + classical tasks.
- Extensible and adaptable architecture for other QPU vendors.
- Improved usability, scheduling, and resource control.
- Extensible to cloud-native environments.

- **Fully open-source** stack with easy installation
- **4 Quantum vendor supports**, 6 access methods
- Partnered with **15 organizations**.
- Deployed to 5 **HPC data centers** (CINECA, RPI and others) and started evaluations
- Enabled **partner's experiments for scientific breakthroughs**.
- Continuously enhancing the stack based on partner feedback

From Integration to Innovation: QRMI and Slurm Driving Quantum Research

“The hardware experiments were managed via the quantum spank plugin for Slurm v0.1.0 [70].”



Jay Gambetta • Following

Director of IBM Research and IBM Fellow

[Visit my website](#)

3mo •

I am delighted to share a major milestone in our journey toward realizing useful quantum computing for chemical simulations.

The team introduced a novel, sample-based method for electronic structure calculations that is both scalable and backed by convergence guarantees. The algorithm, SqDRIFT, employs randomized unitary product formulas to enable quantum Krylov diagonalizations of molecular Hamiltonians.

In a 48-qubit experiment on `ibm_aachen` targeting the ground state of coronene, SqDRIFT achieved an accuracy surpassing that of classical post-Hartree-Fock methods. While Selected Configuration Interaction (SCI), the classical technique that inspired this family of sample-based approaches, still outperforms it, the gap is narrowing. These results, described in the preprint here: <https://lnkd.in/eGCSVE8w>, are highly promising for future quantum-centric supercomputing workflows.

This work was carried out in collaboration with [STFC Hartree Centre](#) through the HNCDI partnership, leveraging the SLURM integration plugin (<https://lnkd.in/eRkVg7kM>) that we co-developed and released earlier this year. We were also honored to collaborate with Ali Alavi, director of the Max Planck Institute for Solid State Research and professor at the University of Cambridge, a pioneer in discrete QMC methods.

Quantum chemistry with provable convergence via randomized sample-based quantum diagonalization

Samuele Piccinelli,^{1,2,*} Alberto Baiardi,¹ Max Rossmannek,¹ Almudena Carrera Vazquez,¹ Francesco Tacchino,¹ Stefano Mensa,³ Edoardo Altamura,^{3,4} Ali Alavi,^{5,4} Mario Motta,⁶ Javier Robledo-Moreno,⁶ William Kirby,⁶ Kunal Sharma,⁶ Antonio Mezzacapo,⁶ and Ivano Tavernelli^{1,†}

¹IBM Quantum, IBM Research Europe - Zurich, CH-8803 Rüschlikon, Switzerland

²Institute of Physics, École Polytechnique Fédérale de Lausanne (EPFL), CH-1015 Lausanne, Switzerland

³The Hartree Centre, STFC, Sci-Tech Daresbury, Warrington, WA4 4AD, United Kingdom

⁴Yusuf Hamied Department of Chemistry, University of Cambridge, Lensfield Road, Cambridge CB2 1EW, United Kingdom

⁵Max Planck Institute for Solid State Research, Heisenbergstr. 1, 70569 Stuttgart, Germany

⁶IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, United States

(Dated: August 5, 2025)

Sample-based quantum diagonalization (SQD) is a recently proposed algorithm to approximate the ground-state wave function of many-body quantum systems on near-term and early-fault-tolerant quantum devices. In SQD, the quantum computer acts as a sampling engine that generates the subspace in which the Hamiltonian is classically diagonalized. A recently proposed SQD variant, Sample-based Krylov Quantum Diagonalization (SKQD), uses quantum Krylov states as circuits from which samples are collected. Convergence guarantees can be derived for SKQD under similar assumptions to those of quantum phase estimation, provided that the ground-state wave function is concentrated, i.e., has support on a small subset of the full Hilbert space. Implementations of SKQD on current utility-scale quantum computers are limited by the depth of time-evolution circuits needed to generate Krylov vectors. For many complex many-body Hamiltonians of interest, such as the molecular electronic-structure Hamiltonian, this depth exceeds the capability of state-of-the-art quantum processors. In this work, we introduce a new SQD variant that combines SKQD with the qDRIFT randomized compilation of the Hamiltonian propagator. The resulting algorithm, termed SqDRIFT, enables SQD calculations at the utility scale on chemical Hamiltonians while preserving the convergence guarantees of SKQD. We apply SqDRIFT to calculate the electronic ground-state energy of several polycyclic aromatic hydrocarbons, up to system sizes beyond the reach of exact diagonalization.

I. INTRODUCTION

Quantum computing has emerged as a novel computational paradigm capable of addressing problems that exhibit unfavorable scaling on classical computers. Quantum chemistry is a prime candidate among these problems. In fact, quantum computers are believed to be able to speed up the solution of the electronic Schrödinger equation within a given basis set, which is a central computational challenge in quantum chemistry [1–3]. Robust quantum algorithms for solving the Schrödinger equation with performance guarantees already exist, and are mostly based on Quantum Phase Estimation (QPE) [4–

of VQE faces scalability challenges due to a steep measurement overhead [13] and the difficulty of navigating the optimization landscape [14]. Alternative strategies are required to scale-up quantum-chemical calculations on quantum computers beyond the reach of brute-force classical simulations – the so-called “quantum-utility” regime [15].

Methods exploiting the quantum computer to sample classically-hard probability distributions are emerging as a promising route towards this goal. Examples of this class of methods for natural-science applications are quantum algorithms inspired by classical selected configuration interaction (SCI) [16–21], such as quan-

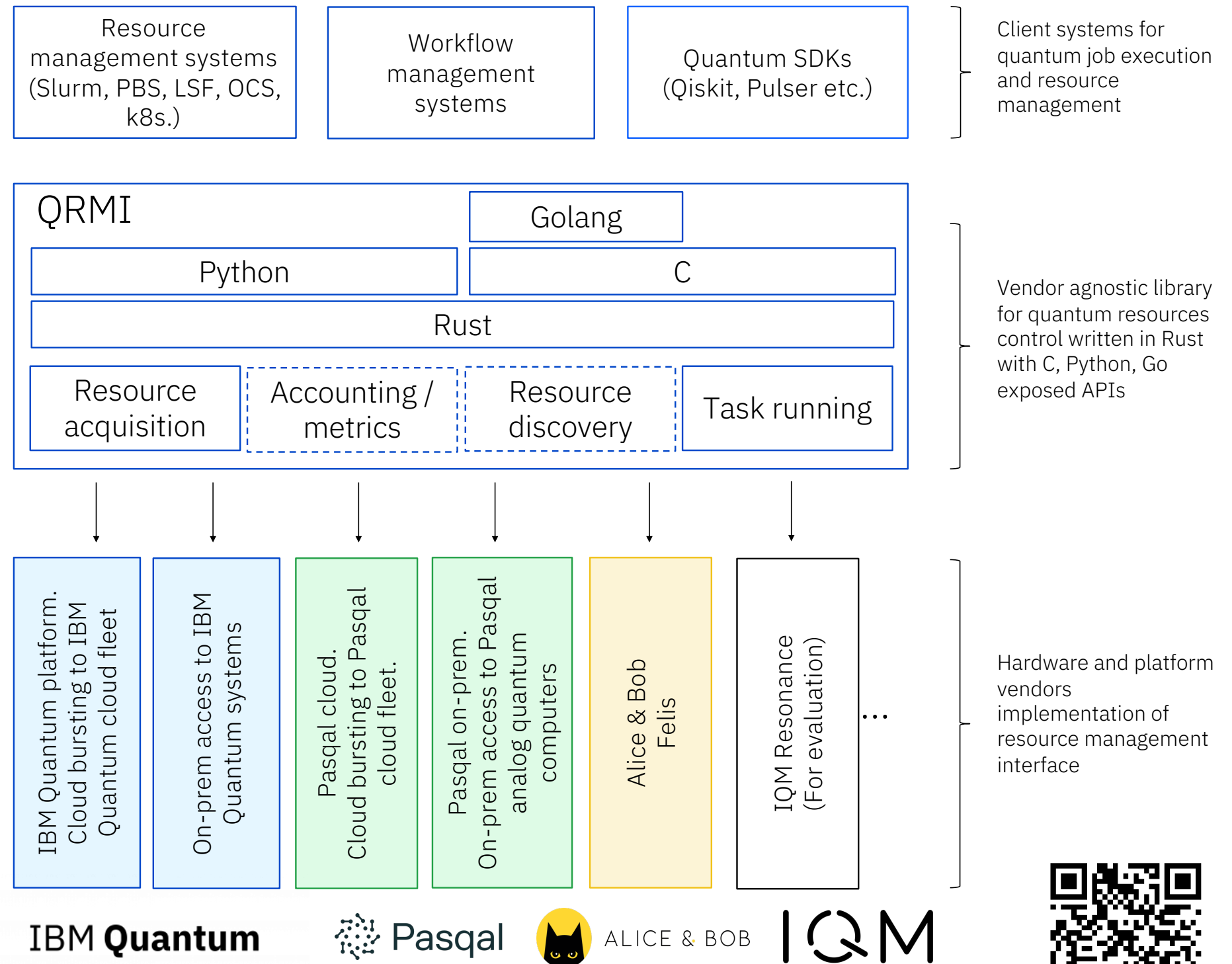
Xiv:2508.02578v1 [quant-ph] 4 Aug 2025

Deployments

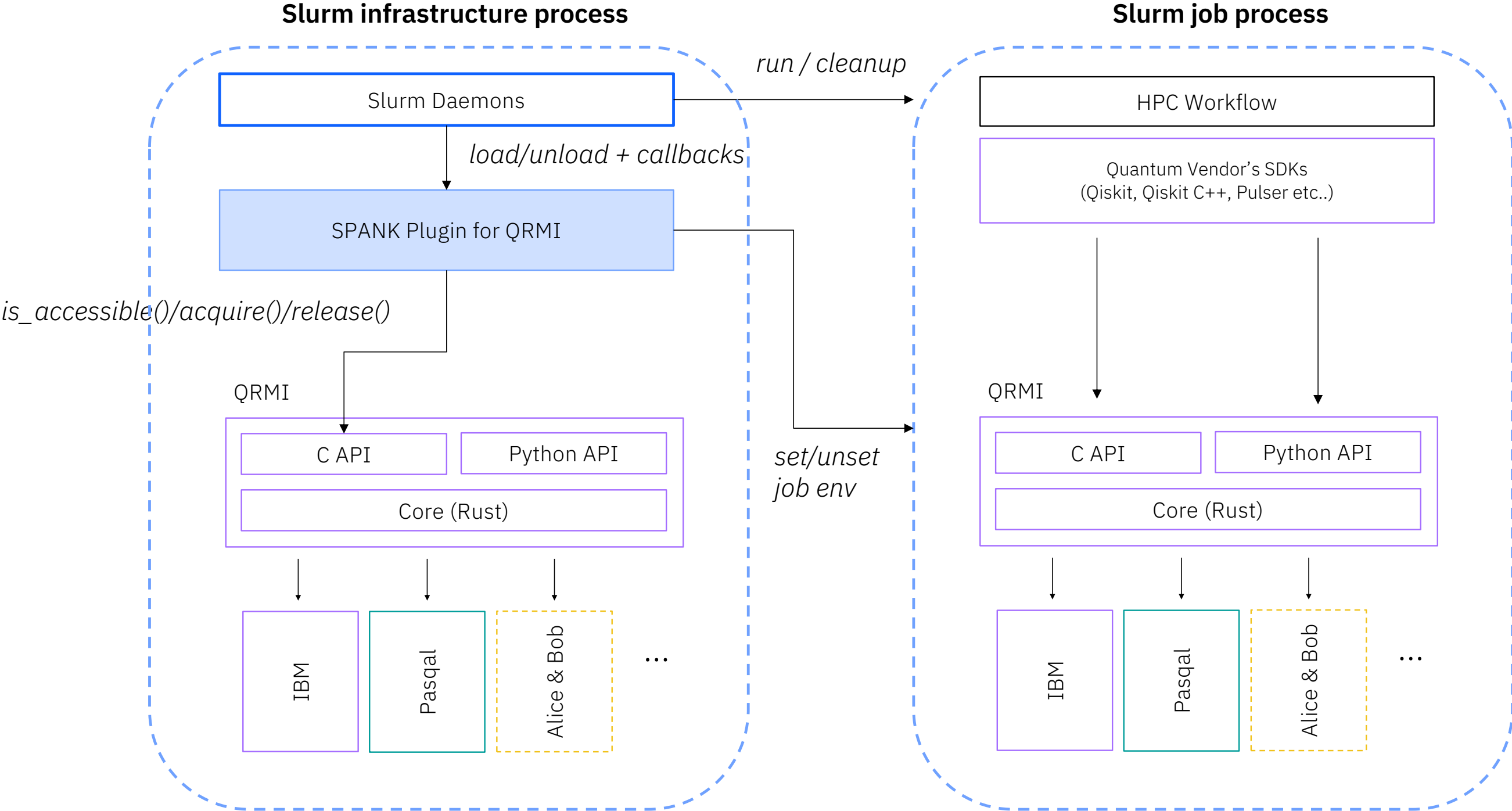


Quantum Resource Management Interface (QRMI)

- Thin and vendor agnostic layer to access, control and monitor underlying on-prem or cloud Quantum systems
- Exposes notion of quantum resources, which might be qubit, entire physical system, virtual QPU, etc.
- 4 sets of APIs available for QRMI:
 - Resource acquisition: availability, acquire, release.
 - Task running: execution on quantum payload on target hardware
 - [2026/4Q] Beta of Accounting, Resource discovery etc.
- Provides loose coupling between clients and systems



Our Slurm Integration



Component Roles & Responsibilities

Slurm

HPC Resource Manager

- Providing unified job submission for quantum + classical tasks
- Job Scheduling - fair share, priority, and partition policies
- Generic resource allocation - heterogeneous resources and accelerators (GRES/TRES, License)
- Admin and security - secrets, resource security and allocation, user authentication

SPANK Plugin

Bridge the gap to QPU resources

- (Slurm Plug-in Architecture for Node and job Kontrol)
- Exposes QPUs as generic resource to Slurm
- Gives Slurm access to the tools to check accessibility, acquire the resources required
- Passes environment variables for connection to QPU resources

QRMI

Manage Quantum Resource

- Accesses resource through API or direct access - customizable
- Acquire the resource for the length of script
- Abstract beyond device specifications and passes any device specific options defined
- Create and monitor quantum jobs

Slurm User Experience

HPC user: use slurm quantum resource

backend is selected in the *slurm resource definition* used for jobs, e.g. *ibm_fez*

resource defined in slurm parameters (configuration)

```
#SBATCH --time=100
#SBATCH --output=<LOGS_PATH>
#SBATCH --gres=qpu:1
#SBATCH --qpu=ibm_fez
#SBATCH --... # other options

srun ...
```

slurm resource is picked up by *application source code* and used for transpilation and execution:

use previously defined resource

```
from qiskit import QuantumCircuit
from qrm primitives import QRMIService
# using an IBM QRMI flavor:
from qrm primitives.ibm import SamplerV2

# define circuit
circuit = ...

# inject credentials needed for accessing the service at this point
load_dotenv()

# instantiate QRMI service and get quantum resource
service = QRMIService()

resources = service.resources()
qrm = resources[0]

# Generate transpiler target and transpile
target = get_target(qrm)
pm = generate_preset_pass_manager(
    optimization_level=1,
    target=target,
)
isa_circuit = pm.run(circuit)

# run the circuit
sampler = SamplerV2(qrm, options={{})
```

IBM Quantum

HPC admin: define access to physical quantum resources

slurm quantum plugin configuration defines which resources are made available for use (e.g. *ibm_fez*)

```
{
  "resources": [
    {
      "name": "ibm_fez",
      "type": "qiskit-runtime-service",
      "environment": {
        "QRMI_IBM_QRS_ENDPOINT": "https://quantum.cloud.ibm.com/api/v1",
        "QRMI_IBM_QRS_IAM_ENDPOINT": "https://iam.cloud.ibm.com",
        "QRMI_IBM_QRS_SERVICE_CRN": "...",
      }
    }
  ]
}
```

In *slurm.conf*, qpu generic resources can be made available to nodes:

```
...
GresTypes=qpu,name
NodeName=node[1-5000] Gres=qpu,name:ibm_fez
...
```



See ux documentation on github for more details
<https://github.com/qiskit-community/spank-plugins/blob/main/docs/ux.md>

QuantumResource

- Defines interfaces to quantum resources. Each Quantum vendor must implement this Rust crate.
- <https://github.com/qiskit-community/qirmi/blob/main/src/lib.rs>



 also used by Slurm plugin

Function	Descriptions
is_accessible	Returns true if device is accessible, otherwise false.
acquire	Acquires quantum resource and returns acquisition token if succeeded. If no one owns the lock, it acquires the lock and returns immediately. If another owns the lock, block until we are able to acquire lock.
release	Releases quantum resource
task_start	Start a task and returns an identifier of this task if succeeded.
task_stop	Stops the task specified by `task_id`. This function is called if the user cancels the job or if the time limit for job execution is exceeded. The implementation must cancel the task if it is still running.
task_status	Returns the current status of the task specified by `task_id`.
task_result	Returns the results of the task.
task_logs	Returns the log messages of the task.
target	Returns a Target for the specified device. Vendor specific serialized data. This might contain the constraints(instructions, properties and timing information etc.) of a particular device to allow compilers to compile an input circuit to something that works and is optimized for a device.
metadata	Returns other specific to system or device data

Moving
toward WLM -
agnostic
software

PBS

A workload manager for job scheduling on HPC clusters, widely used in research and national labs

IBM Spectrum LSF

An enterprise-class workload management and job scheduler for distributed High Performance Computing (HPC) environments

Open Cluster Scheduler(fka Grid Engine)

A distributed resource management system for scheduling jobs across heterogeneous computing environments

Flux

A next-generation HPC workload manager designed for flexible, hierarchical job scheduling on modern supercomputers

Kubernetes

An open-source platform that automates the deployment, scaling, and management of containerized applications

New Preprint Published by the QRMI Community

- Applied a common architecture: each WLM uses its own native resource and lifecycle mechanisms, while QRMI provides a common boundary to heterogeneous quantum resources.
- Extended this approach across PBS, LSF, Open Cluster Scheduler (formerly Grid Engine), Kubernetes, and the Flux Framework.

Examining QRMI as a Unified Interface for Quantum-HPC Integration

Thomas Badts^{††} Tim Boyle^{||} Claudio Carvalho[§] Antonio Córcoles[§] Andrew Damin[×] Vadim Eliseev[§]
Jonathan Frassinetti[†] Daniel Gruber[‡] Hiroshi Horii[§] Eun-Kyung Lee[§] James Machin^{*} Sara Marzella[†]
Mateusz Meller^{ˣˣ} Daniel Milroy[¶] Matthieu Moreau^{††} Munetaka Ohtani[§] Elisabeth Ortega-Carrasco^{††}
Doug Oucharek^{**} Yoonho Park^{§ˣˣˣ} Adarsh Pati^{ˣˣ} Emre M. Sahin^{ˣˣ} Gábor Samu[§] Seetharami Seelam[§]
Amir Shehata^{**} Vanessa Sochat[¶] James Thorne^{||} Oscar Wallis^{ˣˣ} Aleksander Wennersteen^{††}
^{*}Alice & Bob [†]CINECA [‡]HPC Gridware [§]IBM [¶]Lawrence Livermore National Laboratory
^{||}National Quantum Computing Centre ^{**}Oak Ridge National Laboratory ^{††}Pasqal
^{ˣˣˣ}Qilimanjaro Quantum Tech ^{ˣˣ}Rensselaer Polytechnic Institute ^{ˣˣ}Siemens ^{ˣˣ}STFC Hartree Centre
^{ˣˣˣ}Corresponding Author: yoonho@us.ibm.com

Abstract—The efficient and scalable integration of quantum resources into high-performance computing (HPC) environments requires standardized mechanisms for resource management, scheduling, and workflow orchestration across diverse and heterogeneous infrastructures. The Quantum Resource Management Interface (QRMI) addresses this challenge through a thin, vendor-agnostic middleware layer that provides standardized APIs for scheduling, executing, and monitoring quantum workloads while exposing quantum resources as first-class schedulable resources alongside CPUs and GPUs. Although previous work demonstrated QRMI integration with the Slurm workload manager, its applicability across other workload managers remained unexamined. This paper extends the validation of QRMI to a broad range of workload managers, including PBS, LSF, Grid Engine, Kubernetes, and the Flux Framework, encompassing traditional batch schedulers, a cloud-native orchestration platform, and a graph-based scheduler. We examine the integration patterns, implementation requirements, and scheduler-specific considerations associated with each environment and compare QRMI with alternative approaches to quantum resource integration. We demonstrate that QRMI provides a portable and flexible abstraction layer that minimizes scheduler-specific modifications while enabling consistent access to heterogeneous quantum resources across both on-premises and cloud environments.

Index Terms—Quantum Resources, HPC, Job Management, Resource Management, Workload Managers, QRMI, Integration

I. INTRODUCTION

The convergence of quantum and HPC presents unique challenges in integrating quantum systems into existing HPC infrastructure. There is a critical need for standardized interfaces for quantum resource management, particularly as workflow orchestration becomes increasingly important in quantum-HPC integration. Current approaches suffer from fragmentation, making it difficult to deploy quantum computing capabilities across diverse HPC environments.

The Quantum Resource Management Interface (QRMI) introduces a thin and vendor-agnostic middleware layer that abstracts the control and management of quantum resources [1]. It exposes standardized APIs for accessing, scheduling, exe-

cuting, and monitoring quantum workloads, decoupling application logic and resource management from hardware-specific implementations. By treating quantum resources as first-class schedulable resources—alongside CPUs and GPUs—QRMI enables unified visibility and control across heterogeneous systems, while simplifying integration of quantum resources in HPC environments.

Within the broader context of quantum-HPC integration, QRMI plays a central role in enabling hybrid workflows envisioned for Quantum-Centric Supercomputing (QCSC) [3]. As quantum and HPC systems evolve from loosely coupled offload models toward tightly integrated, co-designed platforms, the need for consistent abstractions across heterogeneous resources becomes critical. The QRMI approach has been validated with Slurm. The QRMI APIs allow on-premises and cloud-hosted quantum devices from a variety of hardware vendors to be incorporated into Slurm via extensible interfaces.

In this paper, we examine the QRMI approach across a wider variety of workload managers including PBS (Portable Batch System), IBM Spectrum LSF® (Load Sharing Facility), Grid Engine, Kubernetes, and the Flux Framework. This is, to the best of our knowledge, the first examination of a quantum resource interface spanning traditional batch schedulers (Slurm, PBS, LSF, Grid Engine), a container orchestrator for elastic cloud resources (Kubernetes), and a hierarchical workload manager that uses a graph-based scheduler (Flux Framework). Building on the initial Slurm integration, this paper makes the following contributions:

- Examination of QRMI integrations across five major workload managers: (1) Slurm, (2) PBS, (3) LSF, (4) Grid Engine, and (5) Kubernetes;
- Analysis of integration patterns for batch schedulers and a container orchestrator;
- Quantum integration in a graph scheduler (Flux Framework);
- Suggestions for improvements to QRMI and workload managers to facilitate quantum-HPC integration, practical



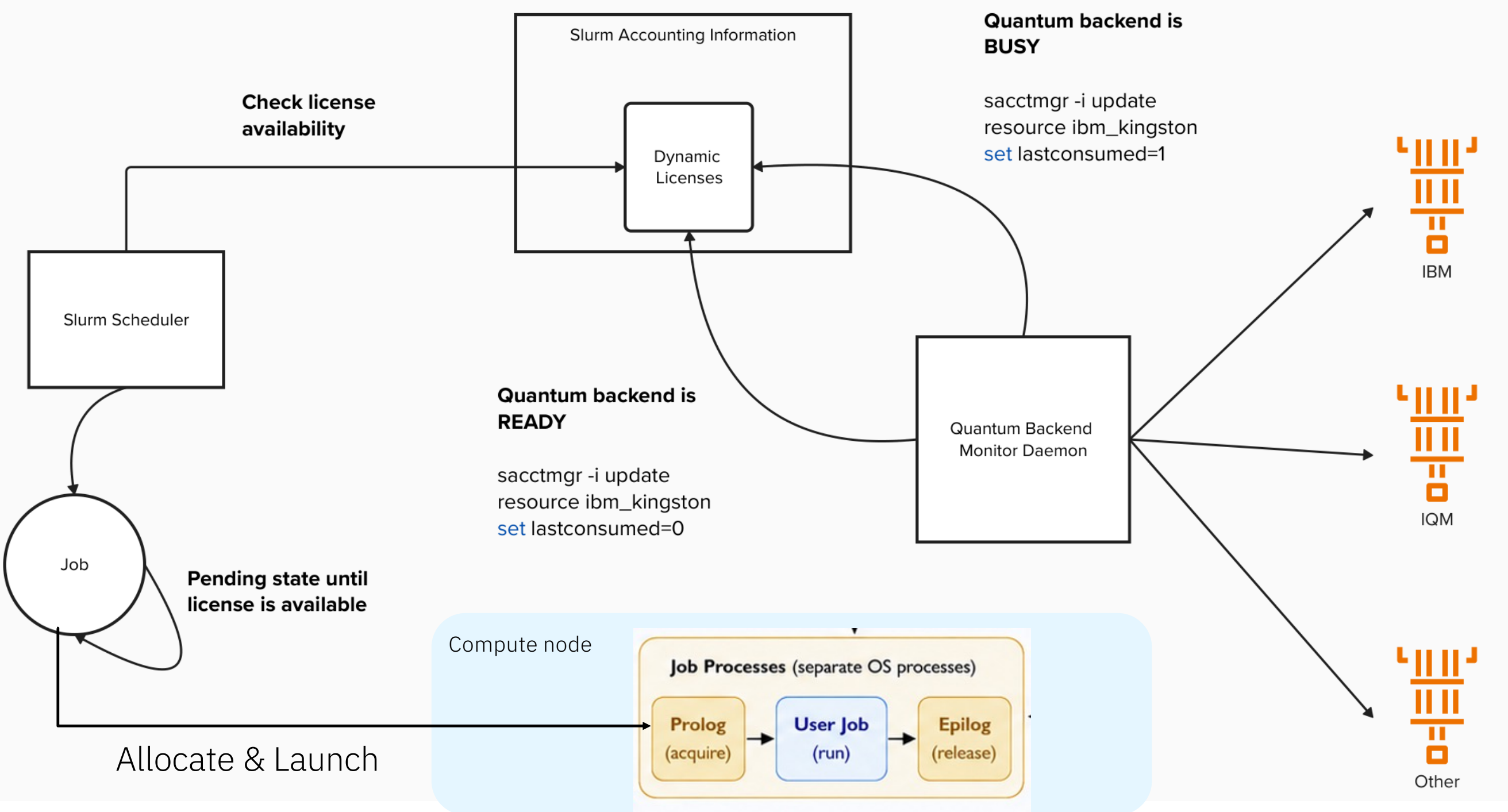
arXiv:2607.19591v1

arXiv:2607.19591v1 [cs.ET] 21 Jul 2026

Recap: Considerations for Working with Quantum Computers

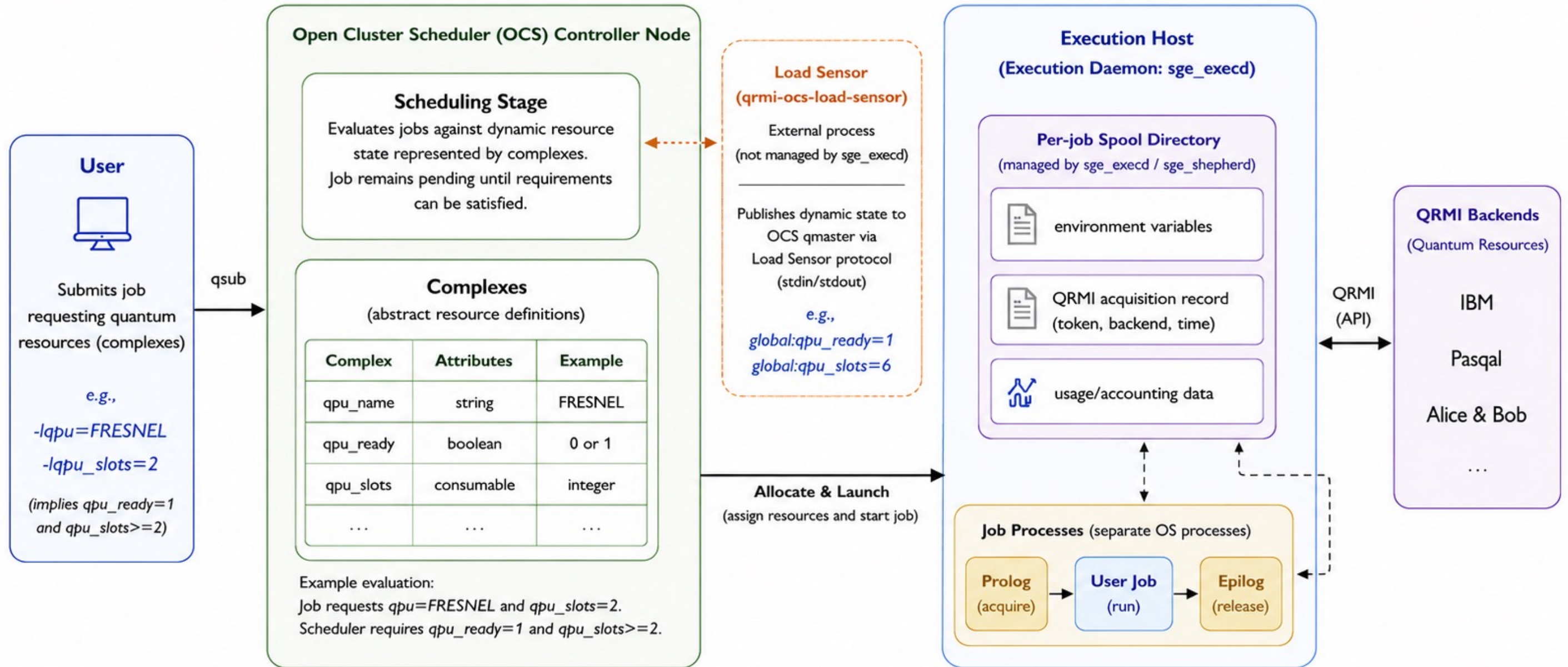
- **Scope** — Can be a node-level resource or a shared, cluster-wide resource.
- **Availability** — Not always available; cannot accept jobs during calibration or other maintenance.
 - Calibration must not run while an HPC job is executing, since transpilation depends on calibration data — if it changes mid-job, transpilation would need to be re-run.
 - A mechanism is needed to block calibration while a job is running.
- **Access** — Mostly REST APIs
- **Authentication** — Accessing a quantum resource requires authentication.
- **Cost** — May be paid resources; usage must be tracked and capped to avoid runaway bills.
- **Queueing** — May have its own queue.

QRMI Integration with Slurm



QRMI integration with Open Cluster Scheduler

(Grid Engine family of schedulers)



Legend:



Controller (OCS)



Execution Host (`sge_execd`)



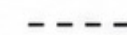
Job Lifecycle Hooks
(prolog/epilog)



External Systems



Communication / Interaction

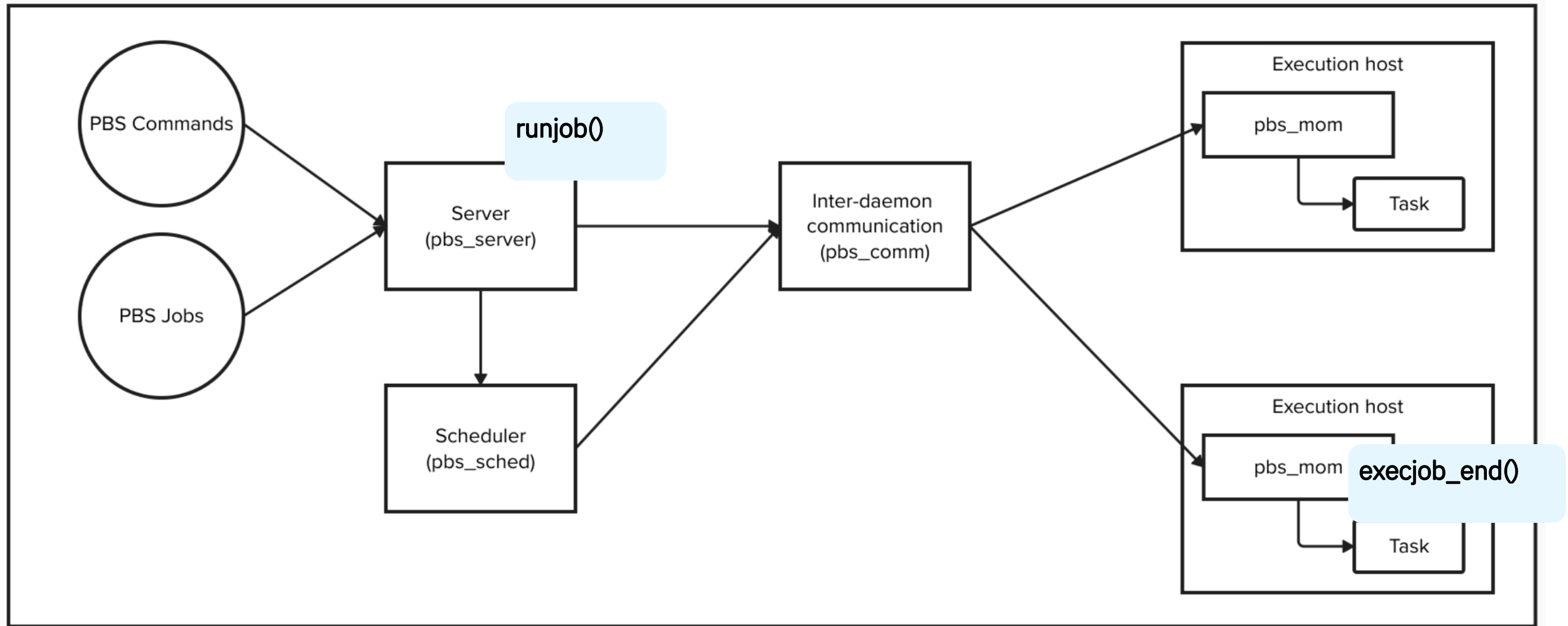


Manages / Persists



Influences Scheduling (dynamic state)

QRMI Integration with PBS – Another way



Future plan

Accounting information support

Retrieve accounting information from quantum resources and make it available through the Quantum SDK/QRMI API, enabling analysis based on raw usage data

Revisiting Acquire/Release

The current implementation can block calibration, but there is no way to know when the resource actually becomes available again.

Credential Management

Currently, users supply credentials such as API tokens via environment variables when running a job. This approach will be reviewed and replaced with a more secure mechanism.

Dynamic Resource Discovery

Advance integration with WLMs and Workflow Managers while revisiting the current evaluation release.

